

Class #11

04.01 ADRs in Practice and Pattern Choice

Software Architectures · Master in Informatics Engineering
Decisions, alternatives, forces, and accepted consequences
Architecture Decision Records (ADRs)

Cláudio Teixeira · claudio@ua.pt

Agenda

- Block 1 - ADR walkthrough 35 min
- Block 2 - ADR 11.1: Sysops Squad migration 45 min
- Block 3 - ADR 11.2: decomposition approach 35 min
- Block 4 - Group ADR rewrite 20 min
- If there is room - ADR 11.3: customer-related functionality 30 min

Block 1

ADR Walkthrough

First: how to read an Architecture Decision Record before we talk about patterns.

What an ADR is

An Architecture Decision Record (ADR) captures one important decision and the reasoning around it.

ADR part	What it must force into the open
Context	Why is this decision necessary now?
Proposed solutions	What real options could we choose?
Technical analysis	What does each option improve, damage, hide, or require?
Final solution	What do we choose, and what flaw do we accept?

Example ADR draft

ADR: Migrate ticket application to microservices

Context

The ticket application handles customer registration, problem ticket entry, ticket routing, reporting, billing, and administration in one deployable.

Production incidents show two recurring symptoms:

- ticket creation slows down when more than 25 customers submit tickets at once
- daytime reporting competes with ticket entry for database resources

Small fixes also require a full application release. The team reports long test runs, high regression risk, and monthly release windows.

Proposed solutions

- keep the monolith
- modularize internally
- split reporting
- migrate to services

Technical analysis

Microservices improve scalability and are the industry standard.

Decision

We will migrate everything to microservices.

Consequences

Teams can deploy independently and the system will scale better.

Why this is a bad example

It has headings. It has alternatives. It still avoids the architecture work.

What looks good

- context exists
- alternatives are listed
- a decision is recorded

What is missing

- actual comparison
- dominant forces
- accepted damage
- evidence to watch

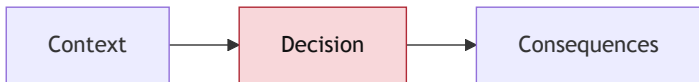
ADRs are not just the decision

They are the memory of the decision.

The decision is only useful if the reasoning is inspectable.

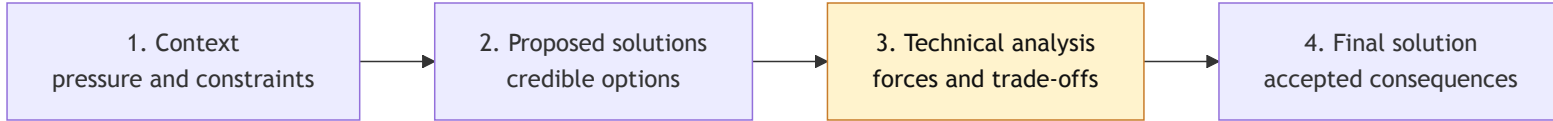
The missing middle

Most weak ADRs jump from context to final solution.



The architecture work lives in the comparison that got skipped.

The four-step version



For this course, the technical analysis is the center of the ADR.

Evaluation forces: deployability · data consistency · operational complexity · fault isolation · team ownership · supportability · reversibility

Evaluation forces

Do not compare options by taste.

- deployability
- data consistency
- operational complexity
- fault isolation
- team ownership
- observability / supportability
- change cost / reversibility

For every ADR: name the dominant forces and the damage accepted.

Source Case

Sysops Squad

Source ADRs: Software Architecture: The Hard Parts, Appendix B

Sysops Squad source context

The source case is a problem-ticket application with several business functions:

- customer registration
- problem ticket entry and processing
- operational and analytical reporting
- billing and payment processing
- administrative maintenance

The recorded pain is scalability, availability, and maintainability.

We will treat these as reference ADRs first, then use the same structure for pattern choices.

ADR 11.1

Migrate Sysops Squad to a Distributed Architecture

Should this monolith remain one runtime, or start moving toward distributed services?

Source ADR: Migrate Sysops Squad Application to a Distributed Architecture, *Software Architecture: The Hard Parts*, Appendix B.

1. Context

Source ADR context:

Sysops Squad is a monolithic problem-ticket application. It supports customer registration, problem ticket entry and processing, operational and analytical reporting, billing and payment processing, and administrative maintenance.

The recorded source pressure is scalability, availability, and maintainability.

Source business case

Change is slow and risky

Developers struggle to locate changes in the large monolith; changes break unrelated behavior.

Testing is not actually healthy

More than 30% of test cases are obsolete or commented out; critical workflow tests are missing.

Deployability blocks agility

Small fixes require full-system deployment, monthly releases, mock deployments, and code freezes.

Faults hurt customer-facing work

The system is not always available; lost tickets and routing issues are hurting customer satisfaction.

Source runtime evidence

Ticket creation pressure

25+

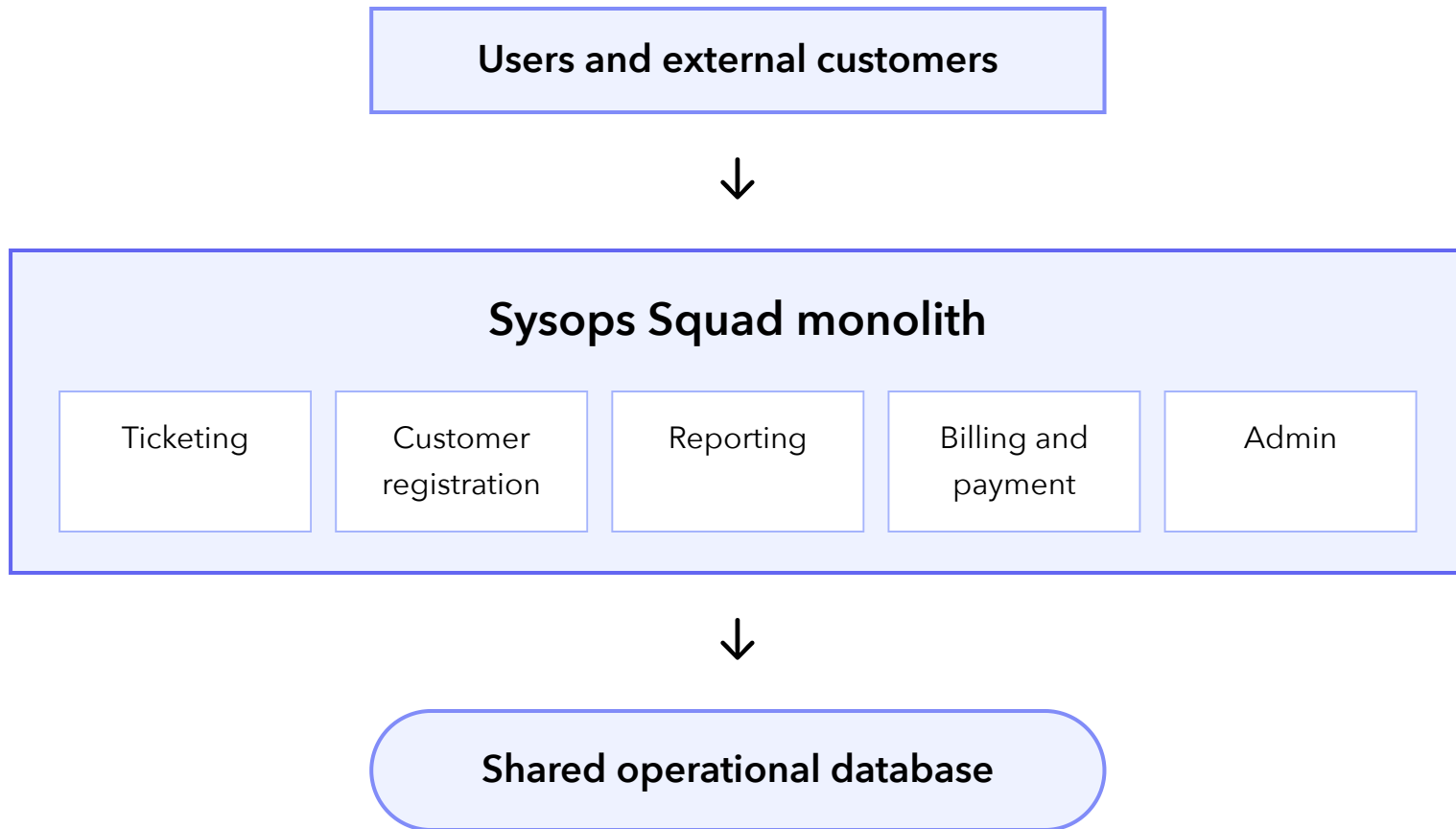
customers creating tickets at the same time can freeze the system.

Reporting load

daytime reports

operational reports run while tickets are entered can also freeze the system.

1. Context map



2. Proposed solutions

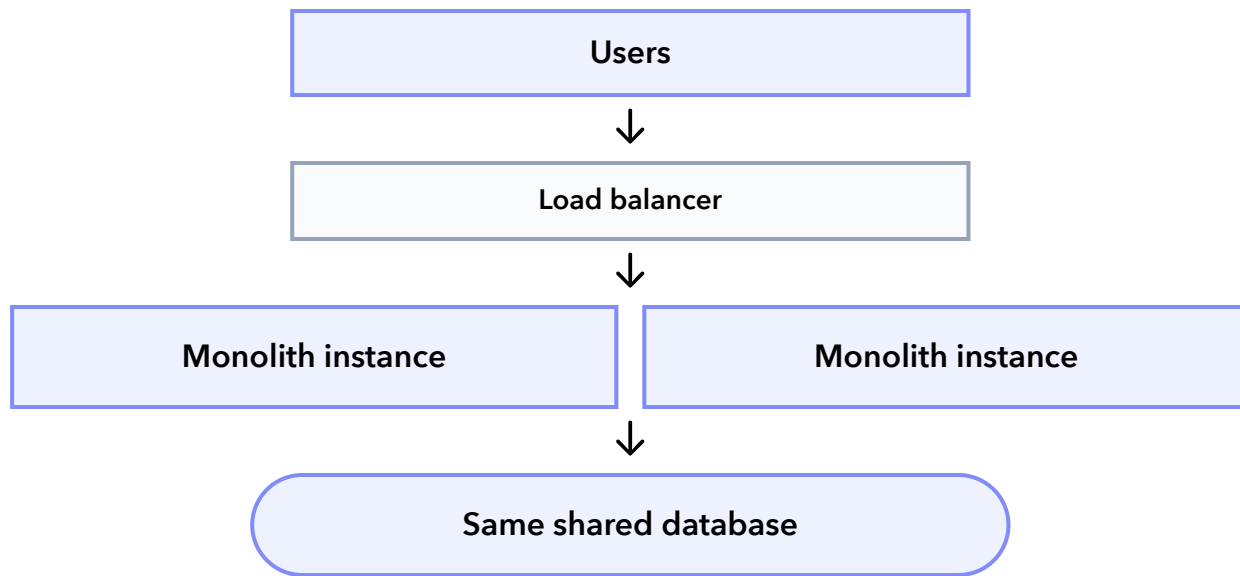
The source ADR does not list a formal option set. For teaching, we reconstruct plausible options from the same pressures.

Option	What it tries to preserve
Keep monolith and optimize	low change risk
Modularize internally	one runtime, clearer code boundaries
Split reporting only	reduce one pressure point
Distributed migration	independent scaling and deployment
Big-bang rewrite	not a distinct architecture; a risky delivery strategy

Option 1

Keep the monolith and optimize

Keep monolith - how it works



Mechanism: add instances, tune the database, cache expensive reads, and improve query paths.

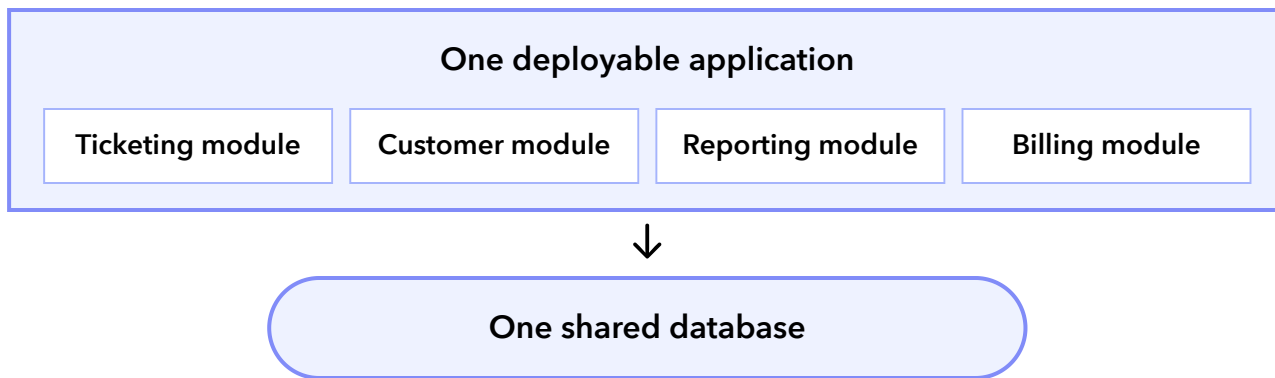
Acceptable when the pressure is mostly resource capacity.

May fail here because ticketing, reporting, billing, and customer flows still share one deployable and one data model.

Option 2

Modularize internally

Internal modularization - how it works



Mechanism: make component boundaries explicit while keeping one build and deployment.

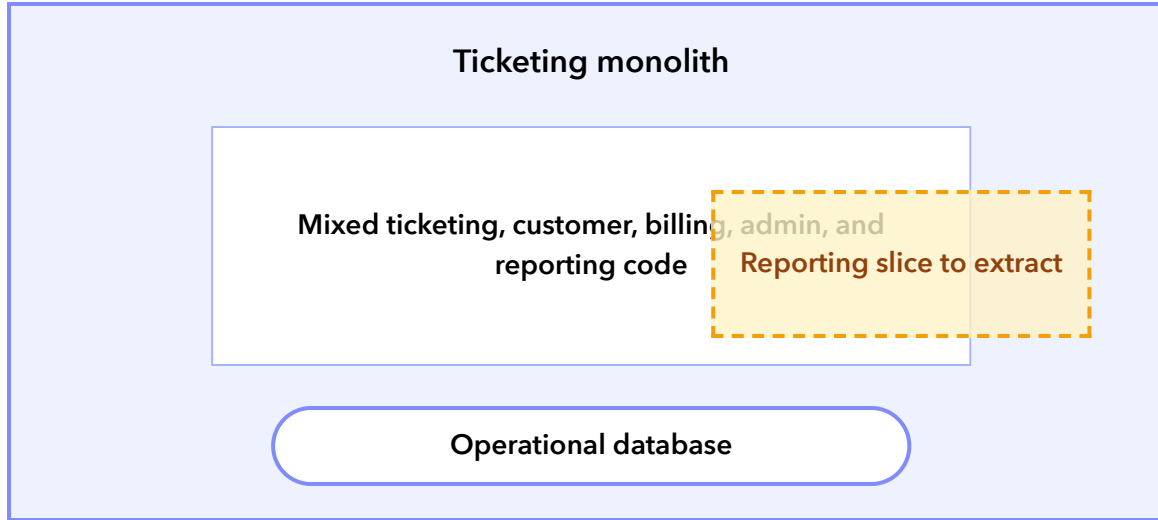
Acceptable when the team needs safer learning before changing runtime topology.

May fail if the main pressure is independent deployment, runtime isolation, or separate scaling.

Option 3

Split reporting only

Reporting split - how it works



Mechanism: move reporting reads to an extractor and reporting store.

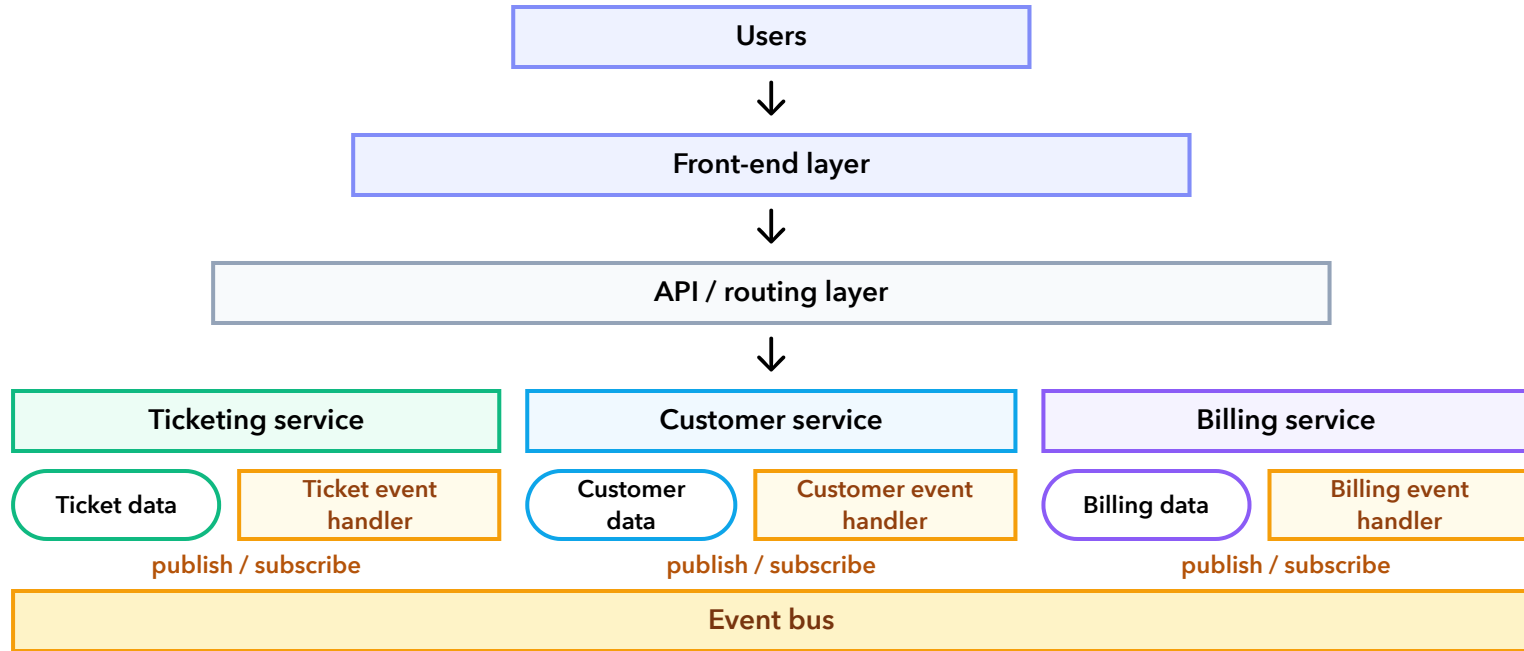
Acceptable when analytical load is the dominant operational pain.

May fail because customer registration, ticketing, billing, and admin remain coupled.

Option 4

Incremental distributed migration

Distributed migration - how it works



Mechanism: route capabilities to separate services and give them explicit data ownership.

Acceptable when availability, scaling, and delivery pressure justify migration cost.

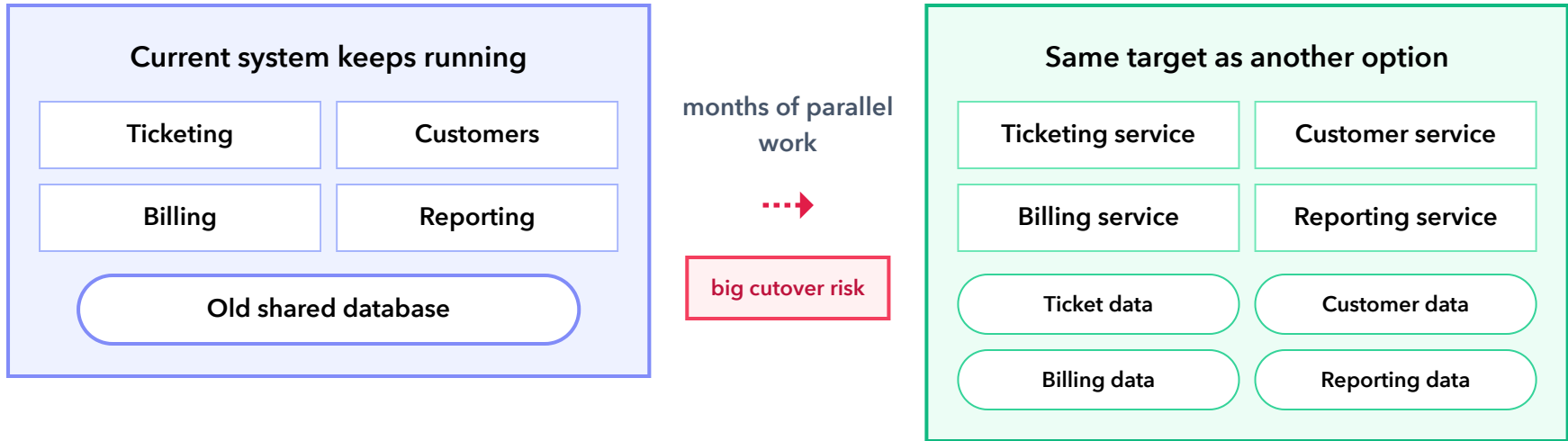
May fail if data ownership, workflow state, contracts, and observability are not designed.

The event bus shown here is an illustrative communication layer for the teaching comparison. The source ADR only commits to a distributed architecture and breaking apart the monolithic database.

Option 5

Big-bang rewrite is not an architecture

Big-bang rewrite - what it really means



Mechanism: choose a target architecture, usually distributed or internally modular, and build it beside the existing system.

This is not a separate architecture option. It is a migration strategy.

May fail because the clean target arrives late, while the old system keeps changing meanwhile.

3. Technical analysis

Option	Works because	May fail because
Keep monolith	lowest migration disruption	shared deploy/runtime/data remain
Internal modularization	dependencies become visible	no independent runtime yet
Split reporting	removes one common load source	leaves core coupling untouched
Distributed migration	improves deployability and fault isolation	creates data/workflow/ops complexity
Big-bang rewrite	may feel clean on paper	not a separate architecture; highest delivery risk

Evaluation forces

For this ADR, the dominant forces are:

- maintainability / changeability
- testability
- deployability
- scalability
- availability / fault isolation
- reporting load isolation

Accepted damage:

- breaking apart the monolithic database
- release and monitoring of multiple deployment units
- migration cost
- delayed feature work during migration

What's the verdict and why?

ADR 11.1

Final solution

Migrate toward a distributed architecture, incrementally.

This is not "microservices are better". It is:

- the monolith is hurting availability, scalability, and delivery
- reporting and ticketing pressures should not freeze the same runtime
- the team accepts migration delay, database breakup, deployment pipeline work, and operations complexity

The decision works only if the migration makes ownership and evidence visible.

Book ADR verdict

The book ADR chooses migration from the existing Sysops Squad monolith to a distributed architecture.

The recorded reasons are availability/fault tolerance for core ticketing, scalability for customer growth and ticket creation, separation of reporting load, faster feature and bug-fix delivery, better testability, and faster deployment.

The recorded consequences are migration delay, extra cost, multiple deployment units to manage, and breaking apart the monolithic database.

The book case closes with the business sponsors accepting the migration business case.

ADR 11.2

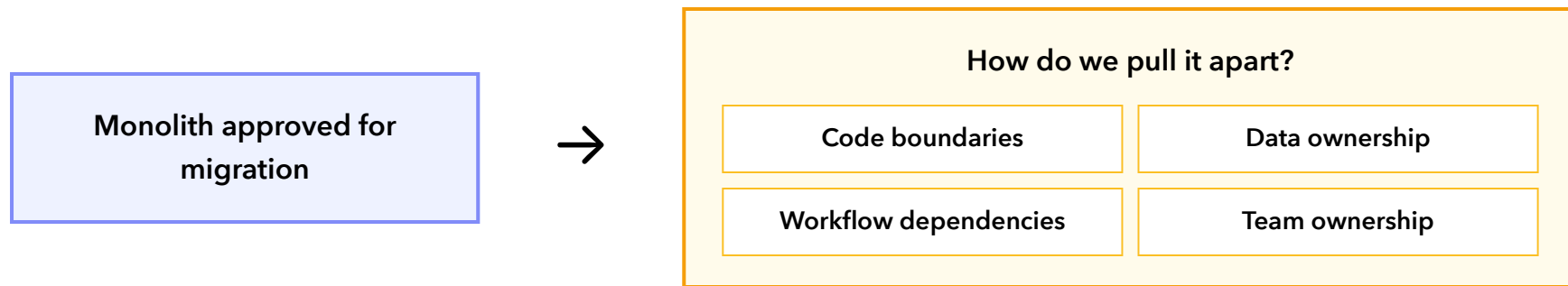
Choose the Decomposition Approach

After deciding to move away from the monolith, how do we start safely?

Source ADR: Migration Using the Component-Based Decomposition Approach, *Software Architecture: The Hard Parts*, Appendix B.

1. Context

The migration decision does not answer where to cut first.



This is a second architecture decision, not implementation detail.

Source feasibility check

The source case does not jump straight to service extraction.

Codebase looks decomposable

The abstractness and instability analysis shows most code lying near the main sequence, with only a few outliers.

Component boundaries already exist

That makes component-based decomposition plausible before service boundaries are fixed.

The decision starts with evidence that the monolith can be pulled apart.

Why not tactical fork first?

Duplicate code and shared
functionality

Shared infrastructure logic: logging
and security

Shared persistence and database calls

Tactical forking would require defining service boundaries up front, before the team has done the slower boundary work.

The source ADR chooses control and learning over extraction speed.

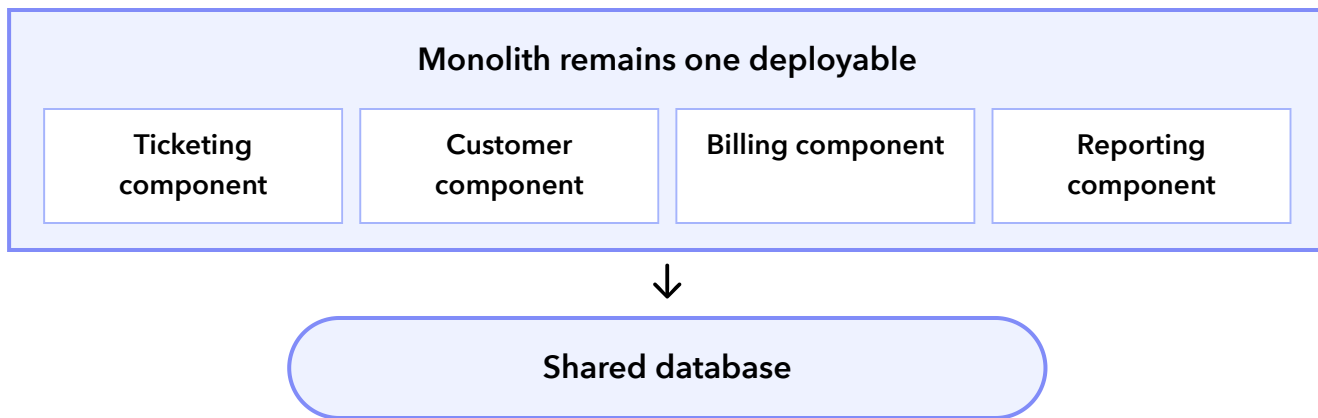
2. Proposed solutions

Option	Basic move
Clean internal components	improve modularity inside the monolith
Component-based decomposition	identify, size, flatten, and depend-map components
Tactical fork	copy a narrow slice to move quickly
Big-bang extraction	split services directly

Option 1

Clean internal components

Internal cleanup - how it works



Mechanism: isolate modules, remove accidental dependencies, and keep the monolith deployable.

Acceptable when reversibility matters more than immediate service autonomy.

May fail if the organization needs independent deployment soon.

Option 2

Component-based decomposition

Component decomposition - how it works

1. Identify components

2. Size components

3. Gather common domain parts

4. Flatten awkward structures

5. Determine dependencies

6. Create component domains

discover what exists

reduce accidental shape

prepare extraction decisions

Mechanism: identify components, size them, gather shared domain parts, flatten awkward structures, then map dependencies.

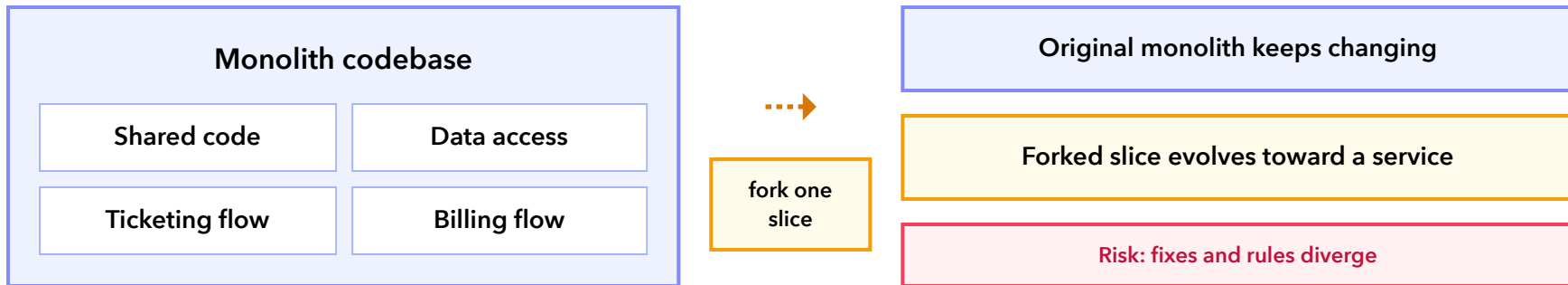
Acceptable when the codebase has meaningful component boundaries to discover.

May fail if the team treats the component map as the final architecture.

Option 3

Tactical fork

Tactical fork - how it works



Mechanism: fork a slice of code and evolve it toward a service path.

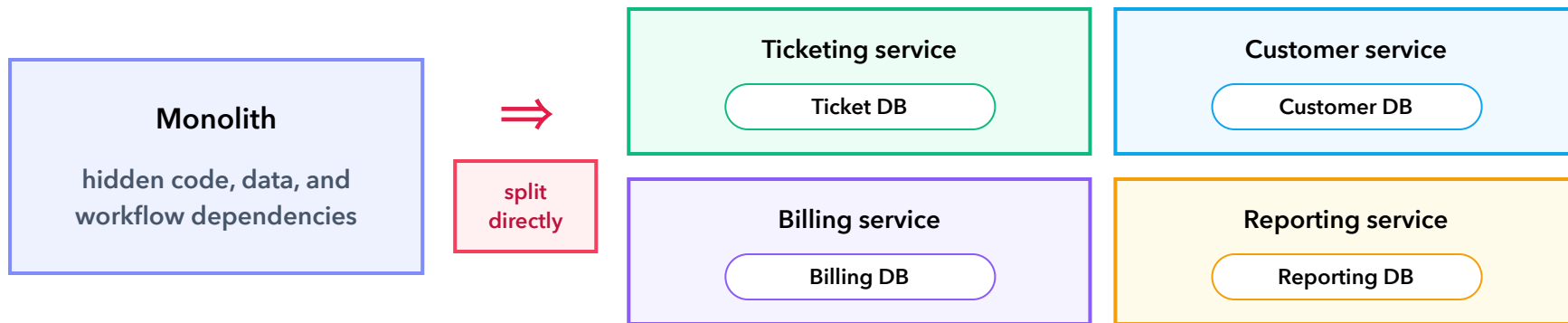
Acceptable only for a narrow, time-boxed extraction.

May fail through divergence, duplicated fixes, and unclear ownership of future changes.

Option 4

Big-bang extraction

Big-bang extraction - how it works



Mechanism: split service boundaries and data stores directly from the monolith.

Acceptable when dependencies are already known and migration risk is low.

May fail because hidden code, data, and workflow dependencies surface after the cut.

3. Technical analysis

Option	Works because	May fail because
Internal cleanup	lowers immediate risk	delays independent deployment
Component decomposition	exposes seams and dependencies	can become analysis without extraction
Tactical fork	fast for one narrow case	divergence and duplicated fixes
Big-bang extraction	clear target structure	late discovery of hidden dependencies

Evaluation forces

Dominant forces:

- reversibility
- team ownership
- change cost
- deployability

Accepted damage:

- slower path to independent deployment
- more preparatory architecture work

What's the verdict and why?

ADR 11.2

Final solution

Prefer component-based decomposition first.

Use tactical fork only for a narrow, time-boxed case.

Do not split services until component dependencies and data ownership are understood.

This decision protects reversibility while the team learns where the real seams are.

Exercise

Rewrite One Assignment Decision as an ADR

20 min

Use a decision your group has already made.

Exercise setup

Pick one architecture decision from your assignment.

Examples of decision areas:

- service boundary
- data ownership
- integration style
- reliability pattern
- documentation / ADR policy
- deployment or operational boundary

Compare the options

For your decision, list:

- the current context
- the technical solutions you considered
- the evaluation forces that matter most
- the final solution you currently prefer

Do not write the final answer first.

Evaluation forces matrix

Use qualitative marks or short notes.

Option	Deployability	Consistency	Operations	Fault isolation	Ownership	Supportability	Reversibility
Option A							
Option B							
Option C							

Output

Each group leaves with:

- one revised ADR
- one rejected alternative
- one consequence or risk to monitor
- one evidence question for the final demo

The ADR is stronger when the trade-off can be inspected.

ADR 11.3

Customer Registration Granularity

Should customer-related functionality be one service, two services, or four services?

Source ADR: Consolidated Service for Customer-Related Functionality, *Software Architecture: The Hard Parts*, Appendix B.

1. Context

Customers must register to access the Sysops Squad support plan.

During registration, the system captures:

Profile information

Credit card information

**Password and security
questions**

Purchased products

The registration API looks like one business action.

The forcing conflict

Product owner pressure

Registration and unsubscribe must be all-or-nothing.

No customer record without matching credit card and password records.

Security pressure

Credit card and password data need stronger access control.

Sensitive data should not be casually reachable by every customer-related operation.

This is transactionality vs security boundary.

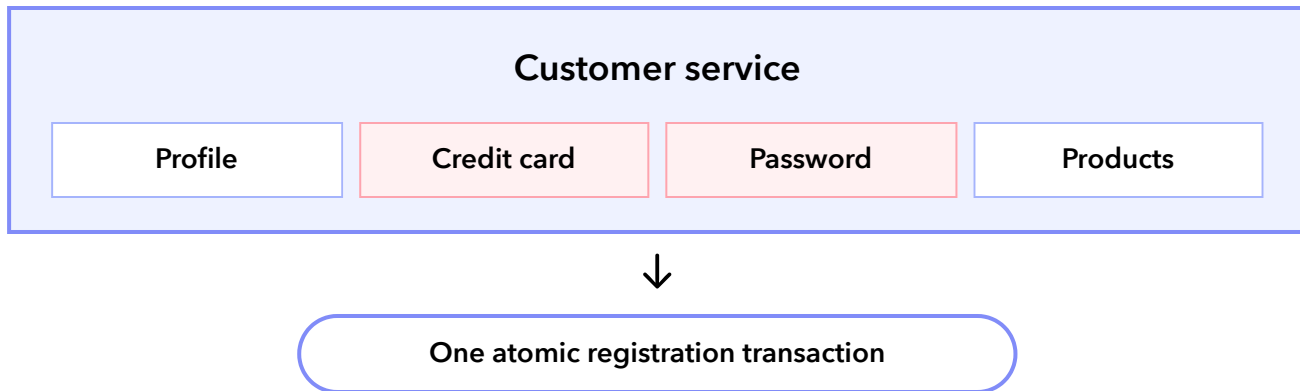
2. Proposed solutions

Option	Basic move
One consolidated customer service	keep profile, products, password, and card together
Two customer services	split non-sensitive and sensitive customer data
Four customer services	split each customer function into its own service

Option 1

One consolidated customer service

Consolidated service - how it works



Mechanism: one service owns the full customer registration boundary.

Works because the all-or-nothing rule can be implemented with one ACID transaction.

May fail if security controls are weak or if the combined service changes and scales too broadly.

Option 2

Two customer services

Profile + secure - how it works



Mechanism: separate sensitive data behind a stronger service boundary.

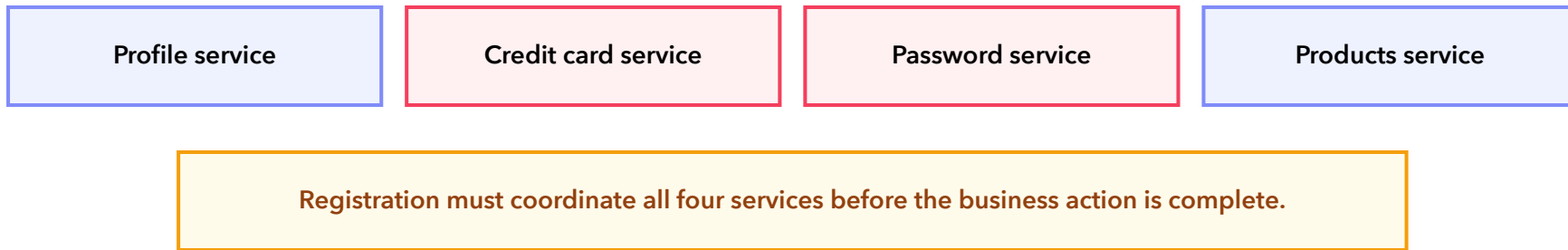
Works because access to password and card operations can be restricted more directly.

May fail because registration now crosses services and loses simple all-or-nothing transactionality.

Option 3

Four customer services

Four services - how it works



Mechanism: every customer function becomes independently deployable.

Works because each capability can have separate ownership and separate access controls.

May fail because the registration workflow becomes a distributed transaction problem for a single user action.

pause

Pause

Tortoise security controls

What "Tortoise" means here

In this book case, "Tortoise" is the organization's custom security library/framework.

It is not a general architecture pattern, and it is not the public Tortoise ORM.

Treat it as a placeholder for concrete controls that already exist in this organization: authentication, authorization, and access-policy checks.

In the book case, credit card data is tokenized and encrypted, and password data is also protected.

The book's decision depends on using those controls at two levels:

API layer

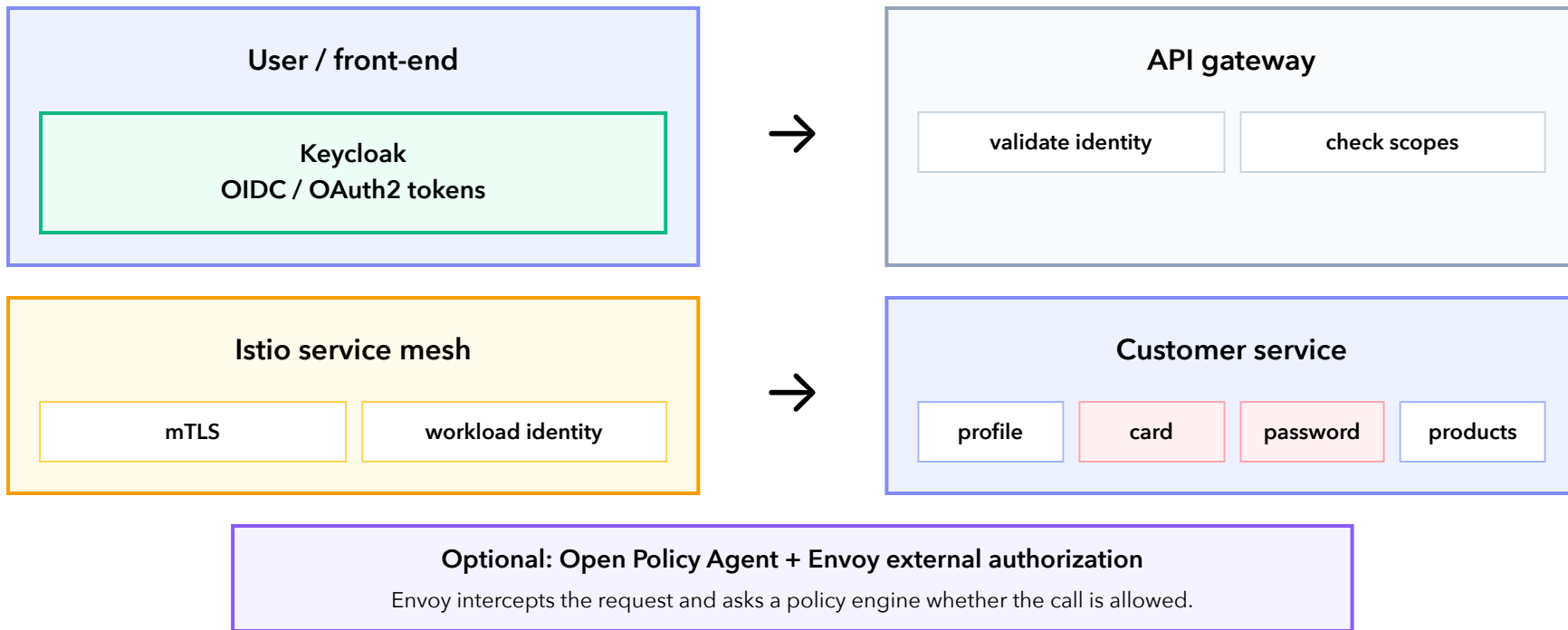
Reject requests before they enter customer functionality.

Service mesh

Check access again on service-to-service communication.

Security is still real, but it is handled by controls around the service boundary.

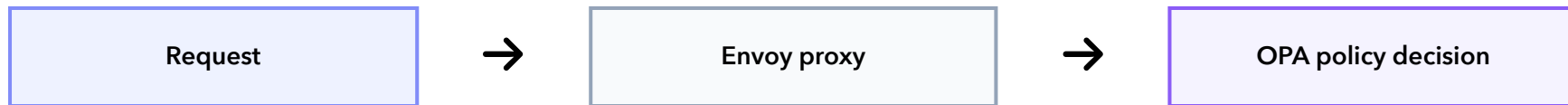
Known substitute controls



What is Open Policy Agent?

Open Policy Agent (OPA) is a general-purpose policy engine.

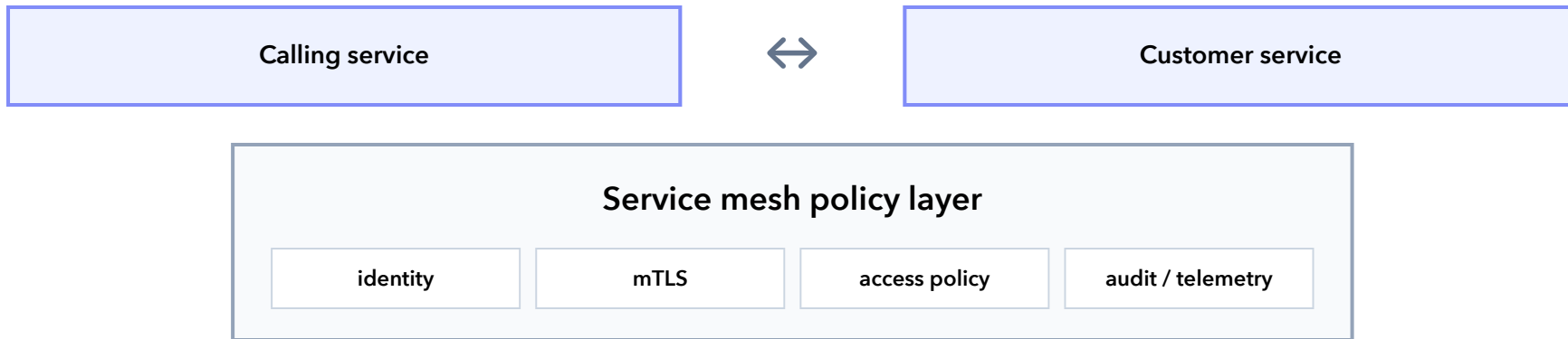
Instead of embedding every authorization rule inside each service, the system can ask OPA for a decision.



In this ADR, OPA is optional. It is a known substitute for richer access-policy decisions, not the main architecture decision.

References: [Keycloak OIDC](#) · [Istio security](#) · [OPA Envoy authorization](#)

What a service mesh adds



This reduces the need to create deployment units only for access control.

It does not remove the need for encryption, tokenization, tests, audit, and policy ownership.

3. Technical analysis

Option	Works because	May fail because
One customer service	keeps registration atomic	wider testing, deployment, and scaling scope
Two services	separates sensitive access	registration becomes coordinated across services
Four services	maximum service separation	heavy workflow coupling for one business action

Evaluation forces

Dominant forces:

- transactionality
- security access control
- deployment risk
- service granularity
- operational complexity

Accepted damage:

- one service must carry broader testing scope
- customer functionality scales as one unit
- security controls must be enforced consistently

What's the verdict and why?

ADR 11.3

Final solution

Use one consolidated customer service.

The all-or-nothing registration requirement dominates the service granularity decision.

Security is handled through tokenization, encryption, Tortoise checks at the API layer, and Tortoise checks through the service mesh.

This is not "security does not matter". It is "security does not automatically mean more services".

Do not split deployment units just because the data has different sensitivity.

Book ADR verdict

The book ADR chooses one consolidated service for profile, credit card, password, and supported products.

Why:

- customer registration and unsubscribe require one atomic unit of work
- separate services would weaken that all-or-nothing guarantee
- Tortoise security controls mitigate access risk around sensitive information

Recorded consequences:

- Tortoise becomes required at API and service-mesh levels
- changes have wider testing and deployment scope
- the combined customer functionality scales as one unit

Revisit

ADR Recap and Home Reading

A compact checklist for judging ADR quality after the lecture. Not another live block.

Good ADR, weak ADR

Good ADRs are not longer. They are more inspectable.

A good ADR usually has

- one decision, not a whole design
- concrete context and constraints
- credible alternatives
- explicit evaluation forces
- technical consequences, including damage
- a decision that explains why now
- follow-up evidence to watch

A weak ADR often has

- a decision already smuggled into the context
- alternatives listed but not compared
- phrases like "industry standard" as proof
- benefits without failure modes
- no accepted trade-off
- no source, status, or later review path
- consequences that read like wishes

The smell: if a future team cannot disagree with the reasoning, the ADR did not expose enough reasoning.

ADR quality recap

Use this quick read before accepting an ADR.

Question	What a strong ADR makes visible
What changed?	The pressure that makes the decision necessary now.
What could we do?	Real options, including the boring or conservative option.
How are we judging?	The evaluation forces that matter in this scenario.
What breaks?	Failure modes, operational burden, and accepted damage.
Why this one?	The reason this option wins despite its flaws.
How will we know?	Evidence, review triggers, or signals that the decision should be revisited.

For this course: the ADR is the place where pattern preference becomes accountable.

Course bibliography links

These are the course-bibliography sources that connect most directly to this lecture.

Decision rationale and trade-offs

- Software Architecture: The Hard Parts: Appendix B gives the Sysops Squad ADRs used here; the surrounding chapters are useful for granularity, decomposition, distributed data, and trade-off analysis.
- Fundamentals of Software Architecture: useful for architecture characteristics, trade-offs, architecture styles, and ADR anti-patterns.
- Software Architecture in Practice: use for quality attributes, tactics, architecture evaluation, and why decisions need evidence.

Description, governance, and boundaries

- ISO/IEC/IEEE 42010:2022: architecture description, stakeholders, concerns, views, and rationale.
- TOGAF Standard, 10th Edition: useful when ADRs sit beside ADM deliverables such as the Architecture Definition Document.
- Learning Domain-Driven Design and Implementing Domain-Driven Design: useful for domain boundaries, bounded contexts, and service-granularity decisions.

Read these with the ADR question in mind: what decision pressure, forces, and accepted consequences does the source help you make explicit?

ADR and pattern references

ADR practice

- Michael Nygard, Documenting Architecture Decisions: lightweight ADR origin story and classic status/context/decision/consequences shape.
- adr.github.io: ADR definitions, templates, practices, and tooling index.
- MADR: Markdown Architectural Decision Records, including a structured template and examples.
- Thoughtworks Technology Radar: Lightweight ADRs: why ADRs belong near the code and remain useful in evolutionary architecture.
- Olaf Zimmermann, Definition of Done for Architectural Decision Making: evidence, criteria, agreement, documentation, and review plan.

Pattern catalogues for later ADRs

- Azure Architecture Center: Cloud Design Patterns: useful for circuit breaker, retry, health endpoint, sidecar, saga, async request-reply, and transactional outbox.
- Microsoft patterns & practices samples: concrete examples for several Azure pattern pages.
- AWS Prescriptive Guidance: Cloud Design Patterns: useful as a cross-check that the pattern is not Azure-specific.
- Google Cloud Architecture Framework: useful for reliability, security, cost, performance, and operational judgement.
- Microservices.io patterns: compact references for transactional outbox, idempotent consumer, saga, and related microservice patterns.

Digest for students: start with the ADR pressure, then use books and catalogues to sharpen the comparison. Do not start by shopping for patterns.